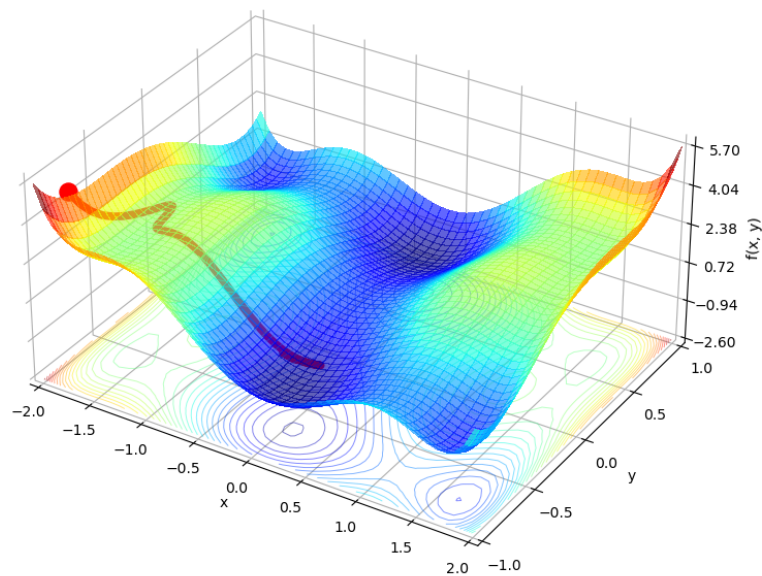


Deep Learning: an Optimization Problem

Bruno Neri

October 7, 2026



Contents

1	Introduction	3
2	First-Order Optimization Methods	3
2.1	Gradient Descent (Batch)	3
2.2	Stochastic Gradient Descent I (SGD)	3
2.3	Stochastic Gradient Descent II (Mini-Batch)	4
2.4	Variance-Reduced Methods (SVRG, SAGA)	4
2.5	Momentum Method	5
2.6	Nesterov Momentum	5
2.7	Adaptive Learning Rate Methods	5
3	Variants of Adam and Recent Optimizers	7
3.1	AMSGrad	7
3.2	Nadam	8

3.3	AdamW	8
3.4	RAdam	8
3.5	AdaBelief	9
3.6	Large-Batch and Memory-Efficient Optimizers	9
3.7	Lion	9
3.8	Matrix-Aware Optimizers (Shampoo, SOAP, Muon, Sophia)	10
4	Techniques Complementing the Optimizer	10
4.1	Learning Rate Schedules	10
4.2	Gradient Clipping	11
4.3	Weight Initialization	11
4.4	Lookahead, SWA and SAM	11
5	Second-Order Methods	12
6	Experiment: a CNN on Fashion-MNIST	12
6.1	Dataset and Model	12
6.2	Experimental Protocol	13
6.3	Results	14
7	Conclusions	18

1 Introduction

Optimization is central to machine learning and deep learning. Training a deep learning model can be formulated as the iterative minimization of an objective function $J(\theta)$ with respect to the model parameters θ ; in this context, $J(\theta)$ is commonly called the **loss function** or **cost function**.

The most widely used optimization methods can be classified into two categories: first-order optimization methods exemplified by the gradient method and second-order (or higher-order) optimization methods, of which Newton’s method is a typical example.

This paper focuses on first-order optimization methods, particularly stochastic gradient descent and its variants. Section 5 briefly reviews second-order methods and discusses the computational factors that limit their use in large-scale deep learning.

The methods are compared in terms of their update rules, computational properties, and practical considerations relevant to optimizer selection and hyperparameter tuning.

The optimization algorithms discussed here are available in deep learning frameworks such as Keras, TensorFlow, and PyTorch. The experiment in Section 6 was conducted in Python using PyTorch.

Notation. At iteration t we denote by $g_t = \nabla_{\theta} J(\theta_t)$ the gradient (or its stochastic estimate) and by η (or α) the learning rate. In all the update rules, products, divisions, squares and square roots between vectors are meant *element-wise*.

2 First-Order Optimization Methods

First-order methods used in machine learning and deep learning are primarily based on gradient descent. This section reviews representative algorithms.

2.1 Gradient Descent (Batch)

Gradient descent iteratively updates the parameter vector θ in the direction opposite to the gradient of the objective function $J(\theta)$, with the aim of reducing its value.

One of the method’s parameters is the learning rate η , which determines the magnitude of the change in θ at each iteration and, consequently, influences the number of iterations required to reach the optimal value of the objective function.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta) \tag{1}$$

The method is straightforward to implement. Under suitable assumptions, convexity of $J(\theta)$ allows convergence to a global minimum. For non-convex objectives, convergence to a global minimum is not generally guaranteed.

Its name derives from the fact that, at each iteration, the algorithm uses all the training set data to compute the gradient $\nabla_{\theta} J(\theta)$.

2.2 Stochastic Gradient Descent I (SGD)

In the method just described, the gradient $\nabla_{\theta} J(\theta)$ is calculated using the entire training set at each iteration. This entails high and redundant computational complexity, in addition to preventing online updates of θ .

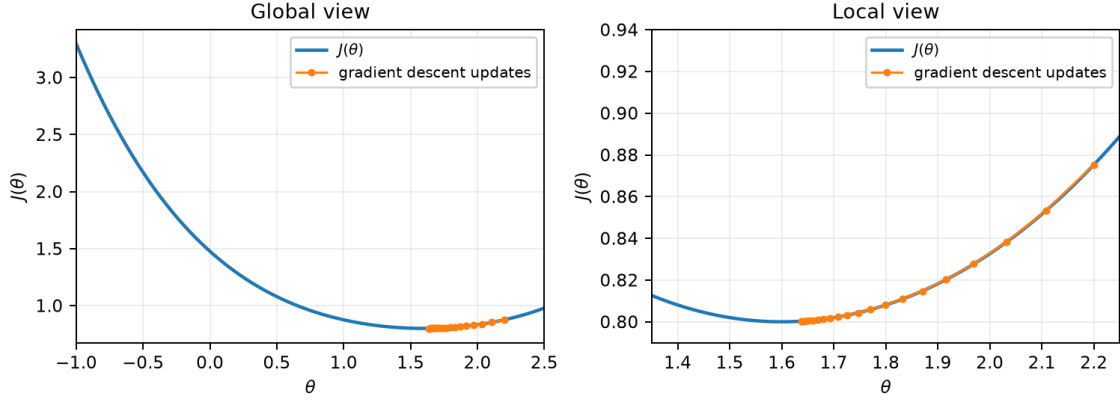


Figure 1: Gradient descent detail.

Stochastic gradient descent (SGD) reduces the per-iteration data requirement by estimating the gradient from a single randomly selected training sample rather than the entire training set.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)}) \quad (2)$$

Using a single sample reduces the computation required for each update; for a linear model, this cost is proportional to the number of features D . The resulting gradient estimate is noisy. This noise can help optimization move away from some saddle points or shallow local minima, but does not guarantee escape from them.

2.3 Stochastic Gradient Descent II (Mini-Batch)

Using individual samples can produce high variance in the gradient direction and substantial variation between consecutive updates.

Mini-batch gradient descent reduces this variance by computing each update from a batch of n training samples. Compared with single-sample SGD, this generally produces more stable gradient estimates, at a higher computational cost per update.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)}) \quad (3)$$

2.4 Variance-Reduced Methods (SVRG, SAGA)

Mini-batching reduces the variance of the gradient estimate at the cost of increased computation per update. With stochastic gradients, residual variance can prevent convergence to the exact minimizer under a fixed learning rate, leaving the iterates in a neighbourhood whose size depends on the gradient variance. Variance-reduced methods address this limitation by constructing an unbiased stochastic gradient whose variance vanishes as θ_t approaches the solution. Here, let $J(\theta) = \frac{1}{N} \sum_{j=1}^N J_j(\theta)$, where J_j is the loss of the j -th training sample.

SVRG [3] periodically stores a snapshot $\tilde{\theta}$ of the parameters and computes the full gradient $\mu = \nabla_{\theta} J(\tilde{\theta})$ on it. Between two snapshots, at each iteration a sample i is drawn and the update is

$$\theta_{t+1} = \theta_t - \eta \left(\nabla_{\theta} J_i(\theta_t) - \nabla_{\theta} J_i(\tilde{\theta}) + \mu \right). \quad (4)$$

The correction term $-\nabla_{\theta}J_i(\tilde{\theta}) + \mu$ has zero mean, so the estimate stays unbiased, and it cancels most of the noise when θ_t is close to $\tilde{\theta}$. For strongly convex functions, this yields linear convergence under standard assumptions.

SAGA [4] avoids the periodic full-gradient computation by storing in a table the last gradient α_j computed for each sample:

$$\begin{aligned}\theta_{t+1} &= \theta_t - \eta \left(\nabla_{\theta}J_i(\theta_t) - \alpha_i + \frac{1}{N} \sum_{j=1}^N \alpha_j \right), \\ \alpha_i &\leftarrow \nabla_{\theta}J_i(\theta_t).\end{aligned}$$

The price is a memory cost of $O(N \cdot D)$ for the table.

Variance-reduced methods have favorable convergence guarantees under specific convexity and smoothness assumptions. In some deep-learning settings, however, their practical benefits have been limited [5]; they are consequently less common in standard deep-learning workflows.

2.5 Momentum Method

Although SGD is widely used, its updates can exhibit slow progress or high variance, depending on the objective and learning-rate choice.

The choice of learning rate affects both convergence speed and stability. Momentum methods address some optimization difficulties by incorporating information from previous gradients; they do not, in general, guarantee avoidance of local minima.

Momentum [1] introduces a velocity-like state v that combines the current gradient with an exponentially weighted contribution from previous gradients. This can accelerate progress in directions with consistent gradients and reduce oscillations in directions with high curvature. The parameter $\beta \in [0, 1)$ controls the decay rate of past gradient contributions.

$$\begin{aligned}v_t &= \beta v_{t-1} - \eta \nabla_{\theta}J(\theta) \\ \theta_{t+1} &= \theta_t + v_t\end{aligned}$$

2.6 Nesterov Momentum

A variant of the momentum method is known as *Nesterov Accelerated Gradient* [2]. The idea behind this variant is to evaluate the gradient not at the current position θ , but rather at the position $\theta + \beta v_{t-1}$ that is, slightly further ahead along the direction of the momentum v_{t-1} . This algorithm also employs a parameter $\beta \in [0, 1)$ which, as in the standard momentum method, determines the decay rate of historical gradient contributions.

$$\begin{aligned}\tilde{\theta}_t &= \theta_t + \beta v_{t-1} \\ v_t &= \beta v_{t-1} - \eta \nabla_{\theta}J(\tilde{\theta}) \\ \theta_{t+1} &= \theta_t + v_t\end{aligned}$$

2.7 Adaptive Learning Rate Methods

Learning-rate selection strongly influences the behavior of SGD. Adaptive methods address this sensitivity by adjusting update magnitudes using statistics of past gradients.

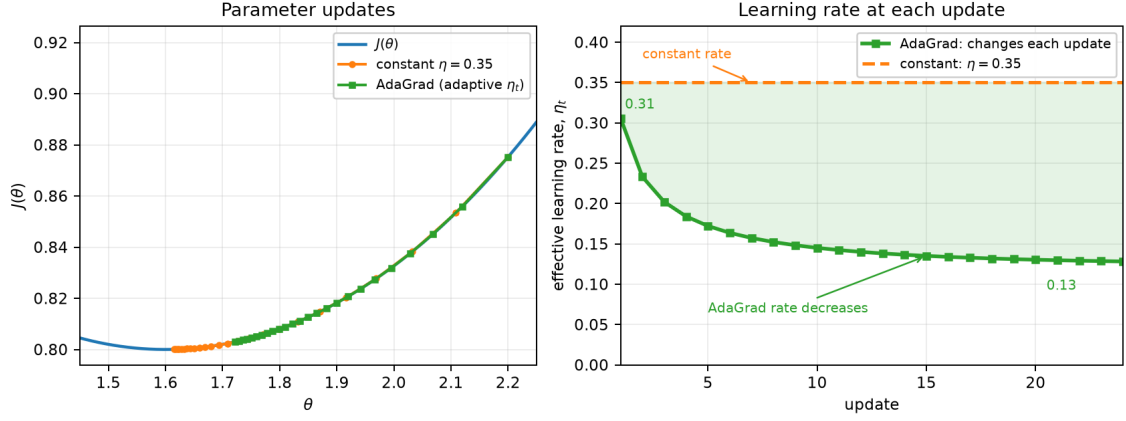


Figure 2: Parameter updates and learning-rate trajectories for a constant learning rate and AdaGrad. The constant rate remains fixed, whereas AdaGrad’s effective rate decreases as squared gradients accumulate.

2.7.1 AdaGrad

AdaGrad [6] adjusts the learning rate using the history of squared gradients.

The difference between AdaGrad and classical gradient descent lies in the fact that, during the update process for θ , the learning rate is not constant; instead, it is continuously recalculated using the historical gradient values accumulated up to the current iteration. A limitation of this procedure is that, with very long training times, the high cumulative value of the gradients drives the learning rate toward zero; consequently, θ stops moving and may settle on a stationary value that is not the optimum.

$$\begin{aligned}
 g_t &= \nabla_{\theta_t} J(\theta_t) \\
 v_t &= \sqrt{\sum_{i=1}^t (g_i)^2 + \epsilon} \\
 \theta_{t+1} &= \theta_t - \eta \frac{g_t}{v_t}
 \end{aligned}$$

2.7.2 RMSProp

RMSProp [7] addresses AdaGrad’s diminishing learning-rate limitation by replacing the cumulative sum of squared gradients with an exponentially decaying average. This average emphasizes recent gradients and estimates the uncentered second raw moment of the gradient.

$$\begin{aligned}
 v_t &= \beta v_{t-1} + (1 - \beta) g_t^2 \quad \text{with } \beta \in [0, 1) \\
 \theta_{t+1} &= \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}
 \end{aligned}$$

2.7.3 AdaDelta

AdaDelta [8], proposed at the same time as RMSProp, uses the same exponentially decaying average of the squared gradients and accounts for the units of the update: in SGD

the update has the units of the gradient, not those of θ . AdaDelta therefore multiplies the normalized gradient by the root mean square of the *previous updates*, which has the same units as θ , reducing reliance on a manually tuned global learning rate. Denoting $\text{RMS}[x]_t = \sqrt{E[x^2]_t + \epsilon}$ and $\rho \in [0, 1)$ the decay rate:

$$\begin{aligned} E[g^2]_t &= \rho E[g^2]_{t-1} + (1 - \rho)g_t^2 \\ \Delta\theta_t &= -\frac{\text{RMS}[\Delta\theta]_{t-1}}{\text{RMS}[g]_t} g_t \\ E[\Delta\theta^2]_t &= \rho E[\Delta\theta^2]_{t-1} + (1 - \rho)\Delta\theta_t^2 \\ \theta_{t+1} &= \theta_t + \Delta\theta_t \end{aligned}$$

2.7.4 Adam

Adam (Adaptive Moment Estimation) [9] combines an exponentially moving average of the gradients, $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$, with an exponentially moving average of the squared gradients. These quantities estimate the first moment and the uncentered second raw moment of the gradient, respectively.

Since m_t and v_t are initialized to zero, they are biased toward zero in the first iterations; the factor α_t in the update compensates for this bias.

$$\begin{aligned} g_t &= \nabla_{\theta_t} J(\theta_t) \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1)g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)g_t^2 \\ \alpha_t &= \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t} \\ \theta_{t+1} &= \theta_t - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

The parameters β_1 and $\beta_2 \in [0, 1)$ for the decay of first- and second-order moments take the default values indicated by the authors in [9], namely $\beta_1 = 0.9$, $\beta_2 = 0.999$, with $\alpha = 10^{-3}$ and $\epsilon = 10^{-8}$.

3 Variants of Adam and Recent Optimizers

Adam has motivated a broad family of optimizers, several of which modify or extend its update rule. This section reviews representative examples. Unless stated otherwise, m_t and v_t are defined as in Adam, and $\hat{m}_t = m_t/(1 - \beta_1^t)$, $\hat{v}_t = v_t/(1 - \beta_2^t)$ denote their bias-corrected versions.

3.1 AMSGrad

Reddi et al. [10] showed that Adam may fail to converge on certain convex problems. Because v_t can decrease over time, the effective learning rate $\alpha/\sqrt{v_t}$ may increase. AMSGrad addresses this behavior by maintaining a running maximum of the second-moment estimate:

$$\begin{aligned}\hat{v}_t^{\max} &= \max(\hat{v}_{t-1}^{\max}, v_t) \\ \theta_{t+1} &= \theta_t - \alpha \frac{m_t}{\sqrt{\hat{v}_t^{\max}} + \epsilon}\end{aligned}$$

so that the per-coordinate learning rate is non-increasing. Many implementations keep the bias correction of Adam in this update.

3.2 Nadam

Nadam [11] incorporates Nesterov momentum into Adam: the update uses a look-ahead of the first moment, as if the next momentum step had already been taken.

$$\theta_{t+1} = \theta_t - \alpha \frac{\beta_1 \hat{m}_t + \frac{(1 - \beta_1)}{1 - \beta_1^t} g_t}{\sqrt{\hat{v}_t} + \epsilon}$$

3.3 AdamW

A common way to regularize a model is to add an L_2 penalty $\frac{\lambda}{2} \|\theta\|^2$ to the loss, which adds the term $\lambda \theta_t$ to the gradient. In SGD this is equivalent to weight decay, but in adaptive methods it is not: the term $\lambda \theta_t$ ends up divided by $\sqrt{v_t}$ like any other part of the gradient, so the weights with large gradients are regularized less. AdamW [12] *decouples* the decay from the gradient and applies it directly to the parameters:

$$\theta_{t+1} = \theta_t - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

AdamW is widely used for Transformer training, although optimizer choice depends on the model, task, and training setup.

3.4 RAdam

In the first iterations v_t is estimated from relatively few gradients, and the variance of the adaptive learning rate can be large. A *warm-up* phase is therefore used in some Adam training setups. RAdam [13] replaces this heuristic with an analytic rectification term. Let

$$\rho_\infty = \frac{2}{1 - \beta_2} - 1, \quad \rho_t = \rho_\infty - \frac{2t\beta_2^t}{1 - \beta_2^t}.$$

If $\rho_t > 4$ the variance is tractable and the adaptive step is rectified by

$$\begin{aligned}r_t &= \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}} \\ \theta_{t+1} &= \theta_t - \alpha r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\end{aligned}$$

otherwise the algorithm falls back on SGD with momentum, $\theta_{t+1} = \theta_t - \alpha \hat{m}_t$.

3.5 AdaBelief

Adam scales the update using the magnitude of the gradient, whereas AdaBelief [14] uses the deviation of the observed gradient from its moving-average prediction m_t . Smaller deviations result in a smaller estimated second moment and, consequently, can produce larger normalized updates.

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2)(g_t - m_t)^2 + \epsilon$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}$$

where $\hat{s}_t = s_t / (1 - \beta_2^t)$.

3.6 Large-Batch and Memory-Efficient Optimizers

LARS and LAMB. With very large batches, selecting one learning rate for the whole network can be difficult when the ratio between update and parameter norms varies substantially across layers. LARS [15] and LAMB [16] address this setting with a layer-wise *trust ratio*. Denoting by $\theta^{(l)}$ and $g^{(l)}$ the parameters and the gradient of layer l , LARS (built on SGD with momentum) rescales the learning rate of the layer by

$$\lambda^{(l)} = \gamma \frac{\|\theta^{(l)}\|}{\|g^{(l)}\| + w\|\theta^{(l)}\|},$$

with γ a trust coefficient and w the weight decay. LAMB applies the same idea to Adam: given the Adam direction with weight decay $r_t^{(l)} = \hat{m}_t^{(l)} / (\sqrt{\hat{v}_t^{(l)}} + \epsilon) + \lambda \theta_t^{(l)}$, the update is

$$\theta_{t+1}^{(l)} = \theta_t^{(l)} - \alpha \frac{\|\theta_t^{(l)}\|}{\|r_t^{(l)}\|} r_t^{(l)}.$$

Adafactor. Adam stores two additional values (m_t and v_t) for each parameter, which triples the memory needed by the parameters. Adafactor [17] reduces this cost by not storing m_t and by *factorizing* v_t : for a weight matrix $W \in \mathbb{R}^{n \times k}$ it keeps only the moving averages of the row sums $R_t \in \mathbb{R}^n$ and of the column sums $C_t \in \mathbb{R}^k$ of g_t^2 , and reconstructs the second moment as

$$\hat{v}_t \approx \frac{R_t C_t^\top}{\mathbf{1}^\top R_t},$$

which requires $O(n + k)$ values instead of $O(nk)$.

3.7 Lion

Lion (EvoLved Sign Momentum) [18] was identified through a program search over optimization algorithms. It maintains a momentum estimate m_t and uses the *sign* of a momentum-based direction in its update, giving each coordinate a step with the same magnitude:

$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_{t+1} = \theta_t - \alpha (\text{sign}(c_t) + \lambda \theta_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t$$

with typical values $\beta_1 = 0.9$ and $\beta_2 = 0.99$. Since the update has unit magnitude for each coordinate, a smaller learning rate and a larger weight decay than those of AdamW are normally required.

3.8 Matrix-Aware Optimizers (Shampoo, SOAP, Muon, Sophia)

All the methods above treat each parameter independently (diagonal preconditioning). A recent line of research exploits the fact that most of the parameters of a neural network are organized in matrices.

Shampoo [19] keeps, for a weight matrix W with gradient G_t , two small preconditioners $L_t = L_{t-1} + G_t G_t^\top$ and $R_t = R_{t-1} + G_t^\top G_t$, and updates

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4},$$

which captures correlations between the rows and the columns of the matrix at a cost much lower than a full second-order method. **SOAP** [20] runs Adam in the eigenbasis of the Shampoo preconditioner.

Muon [21] applies SGD with momentum to the matrix-shaped parameters, but replaces the momentum matrix $M_t = U \Sigma V^\top$ with its orthogonalized version $UV^\top$, approximated with a few Newton–Schulz iterations; the remaining parameters (embeddings, biases, output layer) are usually trained with AdamW.

Sophia [22] preconditions the momentum with a cheap, periodically updated estimate of the diagonal of the Hessian, and clips the update element-wise to limit the effect of a bad estimate.

These optimizers are an active research topic, and their relative advantages depend on the model and on the scale of training; the reader is referred to the original works.

4 Techniques Complementing the Optimizer

Training outcomes depend on the optimizer and on additional choices that are not part of the update rule itself.

4.1 Learning Rate Schedules

Even with an adaptive optimizer, decreasing the global learning rate during training is often useful: larger steps can support early progress, whereas smaller steps can support later refinement. Let η_0 be the initial learning rate and T the total number of iterations. Common schedules include:

$$\begin{aligned} \text{step decay: } \eta_t &= \eta_0 \gamma^{\lfloor t/s \rfloor}, \quad \gamma \in (0, 1) \\ \text{cosine annealing [23]: } \eta_t &= \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T} \right) \\ \text{linear warm-up [24]: } \eta_t &= \eta_0 \frac{t}{T_w}, \quad t \leq T_w \end{aligned}$$

In the step decay, the learning rate is multiplied by γ every s iterations. A warm-up phase lasting T_w iterations can be useful with large batches or during the early iterations of some adaptive methods; it is often followed by a decay schedule. Warm-up combined

with cosine annealing is one schedule used for Transformer training. Cyclical schedules [25], which make the learning rate oscillate between a minimum and a maximum value, and the related *one-cycle* schedule are also used in practice.

4.2 Gradient Clipping

In regions where the gradient norm is large, a single update may produce an excessively large parameter change. Such gradients can occur, for example, when training recurrent networks. Norm clipping [26] rescales the gradient when its norm exceeds a threshold c :

$$g_t \leftarrow g_t \cdot \min \left(1, \frac{c}{\|g_t\|} \right).$$

The direction of the gradient is preserved, only its length is limited.

4.3 Weight Initialization

The initialization θ_0 affects activation and gradient scales during early training. For a layer with n_{in} inputs and n_{out} outputs, Glorot (Xavier) initialization [27] draws the weights with variance

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}},$$

which suits sigmoid and tanh activations, while for ReLU activations He initialization [28] uses $\text{Var}(w) = 2/n_{\text{in}}$. Both choices keep the variance of the signal approximately constant from layer to layer.

4.4 Lookahead, SWA and SAM

Some techniques can be combined with a base optimizer and are intended to affect training stability or generalization; their effects depend on the method and experimental setting.

Lookahead [29] keeps two sets of weights. The “fast” weights θ are updated by the base optimizer for k steps, then the “slow” weights ϕ move toward them:

$$\phi \leftarrow \phi + \alpha(\theta_k - \phi), \quad \theta \leftarrow \phi.$$

Stochastic Weight Averaging (SWA) [30] averages the weights visited by SGD in the last part of training, $\theta_{\text{SWA}} = \frac{1}{n} \sum_{i=1}^n \theta_i$. In some reported settings, this produces flatter minima and improves generalization.

Sharpness-Aware Minimization (SAM) [31] explicitly looks for flat minima by minimizing the worst-case loss in a neighbourhood of radius ρ :

$$\min_{\theta} \max_{\|\varepsilon\| \leq \rho} J(\theta + \varepsilon), \quad \hat{\varepsilon} = \rho \frac{\nabla_{\theta} J(\theta)}{\|\nabla_{\theta} J(\theta)\|},$$

and the update uses the gradient computed at the perturbed point $\theta + \hat{\varepsilon}$. SAM requires two gradient computations per iteration.

5 Second-Order Methods

Second-order methods use the curvature of $J(\theta)$, described by the Hessian matrix $H_t = \nabla_{\theta}^2 J(\theta_t)$. The prototype is Newton’s method:

$$\theta_{t+1} = \theta_t - \eta H_t^{-1} g_t.$$

The step is rescaled according to curvature in each direction, and Newton’s method can converge rapidly near a suitable minimum. For a network with D parameters, storing H_t requires $O(D^2)$ memory, while direct inversion requires $O(D^3)$ operations; these costs are prohibitive when D is very large. Moreover, for non-convex objectives, H_t need not be positive definite, and an unmodified Newton step may point toward a maximum or a saddle point. Practical methods therefore approximate or avoid explicitly forming H_t :

- **Quasi-Newton methods (BFGS, L-BFGS).** They build an approximation B_t of H_t^{-1} using only the differences $s_t = \theta_{t+1} - \theta_t$ and $y_t = g_{t+1} - g_t$ [32]:

$$B_{t+1} = \left(I - \rho_t s_t y_t^\top\right) B_t \left(I - \rho_t y_t s_t^\top\right) + \rho_t s_t s_t^\top, \quad \rho_t = \frac{1}{y_t^\top s_t}.$$

L-BFGS stores only the last m pairs (s_t, y_t) , with a memory cost of $O(mD)$. It can be effective in full-batch settings, but its performance may be sensitive to mini-batch noise.

- **Hessian-free optimization** [33]. It never forms H_t : the products $H_t v$ are computed by automatic differentiation, and the system $H_t d = -g_t$ is solved approximately with the conjugate gradient method.
- **K-FAC** [34]. It approximates the Fisher information matrix of each layer, a block-diagonal substitute of the Hessian, with a Kronecker product $A \otimes G$ of two small matrices (the covariances of the layer inputs and of the back-propagated gradients), whose inverse is simply $A^{-1} \otimes G^{-1}$.

The computational cost and implementation complexity of these methods help explain the continued prevalence of first-order methods. The matrix-aware optimizers in Section 3 incorporate some curvature-related structure while retaining different computational trade-offs from full second-order methods.

6 Experiment: a CNN on Fashion-MNIST

This section compares a subset of the optimizers discussed in Sections 2 and 3 by training FashionCNN on Fashion-MNIST. The experiment examines the behavior of different update rules under a shared model, initialization, and data protocol; it is not designed to identify the best achievable accuracy.

6.1 Dataset and Model

Fashion-MNIST [35] consists of 60 000 training and 10 000 test images, 28×28 pixels in grayscale, of 10 classes of clothing items (Figure 3). Raw pixel values $p \in \{0, \dots, 255\}$ are converted to float32 and normalized as $x = (p/255 - 0.2860)/0.3530$, using fixed, rounded

training-set statistics. The model receives tensors of shape $B \times 1 \times 28 \times 28$; no resizing or data augmentation is used. The sample images in Figure 3 show the original grayscale pixels, before normalization.

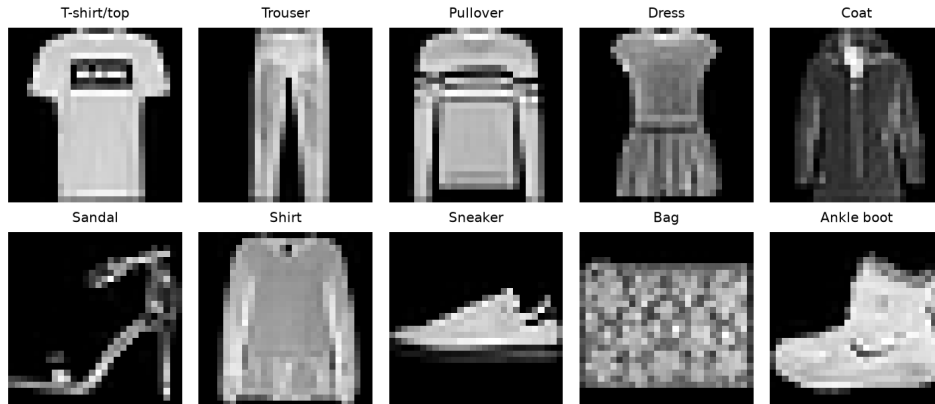


Figure 3: One example for each of the ten classes of Fashion-MNIST.

The implementation uses FashionCNN, with 304 682 trainable parameters, including the learnable scale and shift of batch normalization, and the following structure:

1. three convolutional blocks with 32, 64 and 128 channels. Each block contains two bias-free 3×3 convolutions (padding 1), each followed by batch normalization and ReLU, then max pooling 2×2 and spatial dropout with probability 0.1. The spatial dimensions change from $28 \rightarrow 14 \rightarrow 7 \rightarrow 3$;
2. global average pooling, reducing the final $128 \times 3 \times 3$ representation to 128 features;
3. fully connected layer $128 \rightarrow 128$ with ReLU and dropout with probability 0.4;
4. fully connected layer $128 \rightarrow 10$, producing unnormalized logits for the cross-entropy loss; softmax is not applied before this loss.

Training and evaluation are implemented with PyTorch Lightning. Training loss is the example-weighted mean of mini-batch cross-entropies measured before each parameter update, with the model in training mode. Evaluation loss is also example-weighted, and accuracy is the total number of correct predictions divided by the number of evaluated images. Evaluation uses the weights at the end of the epoch: dropout is disabled, and batch normalization uses running statistics rather than the current mini-batch statistics. In the final runs, the official test set is supplied to Lightning’s validation loop, so the saved `eval_loss` and `eval_acc` are test metrics despite the validation naming in the code. In the sweep, the same loop instead evaluates the held-out training subset. The experiment runner does not invoke the model’s separate `test_step` or export its confusion matrix.

6.2 Experimental Protocol

Thirteen optimizers are compared: SGD, Momentum, Nesterov, AdaGrad, AdaDelta, RMSProp, Adam, AMSGrad, AdamW, Nadam, RAdam, AdaBelief and Lion. Eleven use the implementations provided by `torch.optim`; AdaBelief and Lion are implemented in the notebook as subclasses of `torch.optim.Optimizer`. Momentum and Nesterov use

PyTorch SGD, AMSGrad uses PyTorch Adam with `amsgrad=True`, and AdamW uses PyTorch AdamW with weight decay 10^{-4} . To make the comparison as fair as possible:

- for each random seed all the optimizers start from the same initial weights and see the mini-batches in the same order; mini-batches have 128 images, except for the last batch, which has 96 images in final training;
- every optimizer is trained for 30 epochs on all 60 000 training images, and training is repeated with seeds 0, 1 and 2. This gives 469 mini-batches per epoch and 14 070 optimizer updates per run;
- a reference learning-rate parameter η_0 is chosen separately for each optimizer with a short *sweep*: the model is trained for 4 epochs on the first 20 000 training images and the loss is measured on the last 10 000 images of the training set, using seed 123. These subsets do not overlap. The grid starts with three values spaced by a factor of about 3 and is extended by a factor 3, at most three times, when the best value lies on its edge. The candidate with the lowest final sweep validation loss is selected. The test set is *never* used to choose this parameter;
- OneCycleLR is applied to every optimizer, both during the sweep and during final training, with peak learning rate $\eta_{\max} = 10\eta_0$. Here η_0 denotes the sweep-selected reference parameter, while the schedule’s initial rate is $\eta_{\text{initial}} = \eta_{\max}/25 = 0.4\eta_0$ and the target minimum is $\eta_{\min} = \eta_{\text{initial}}/10^4 = 4 \cdot 10^{-5}\eta_0$. The two-phase schedule uses cosine annealing, increasing for approximately the first 30% of updates and decreasing thereafter. It is stepped after every optimizer update and spans the corresponding training budget. Momentum cycling is disabled for all custom optimizers, so their momentum coefficients remain fixed;
- the remaining hyperparameters are $\beta = 0.9$ for Momentum, Nesterov and RMSProp, $\rho = 0.9$ for AdaDelta, $(\beta_1, \beta_2) = (0.9, 0.999)$ for the Adam family and AdaBelief, and $(0.9, 0.99)$ for Lion. AdaBelief uses $\epsilon = 10^{-16}$; AdamW uses weight decay 10^{-4} and Lion uses decoupled weight decay 0.1. SGD uses momentum $\beta = 0$. Training uses float32 precision, without gradient clipping or early stopping.

The selected η_0 values are 0.1 for SGD, 0.01 for Momentum, 0.03 for Nesterov, approximately 0.00333 for AdaGrad, 0.1 for AdaDelta, 0.0003 for RMSProp, 0.001 for Adam, AMSGrad and AdamW, 0.0003 for Nadam and AdaBelief, 0.003 for RAdam, and 0.0001 for Lion. For example, $\eta_0 = 0.0003$ gives an initial rate of 0.00012 and a peak of 0.003. AdaGrad and AdaDelta require extending their initial grids toward smaller values. Nadam has the lowest observed sweep validation loss (0.251), but this does not imply the highest final test accuracy. Each sweep trial runs a complete four-epoch OneCycleLR schedule, rather than the first four epochs of the final 30-epoch schedule; short-run selection is therefore an approximation to the final optimization setting.

6.3 Results

Table 1 reports the results for FashionCNN after epoch 30. Accuracy, training loss and test loss are means over three seeds; accuracy is reported with the sample standard deviation, not a confidence interval. The learning-rate column contains the sweep-selected η_0 , not the initial rate or the OneCycleLR peak. Figures 5 and 6 show the behaviour during

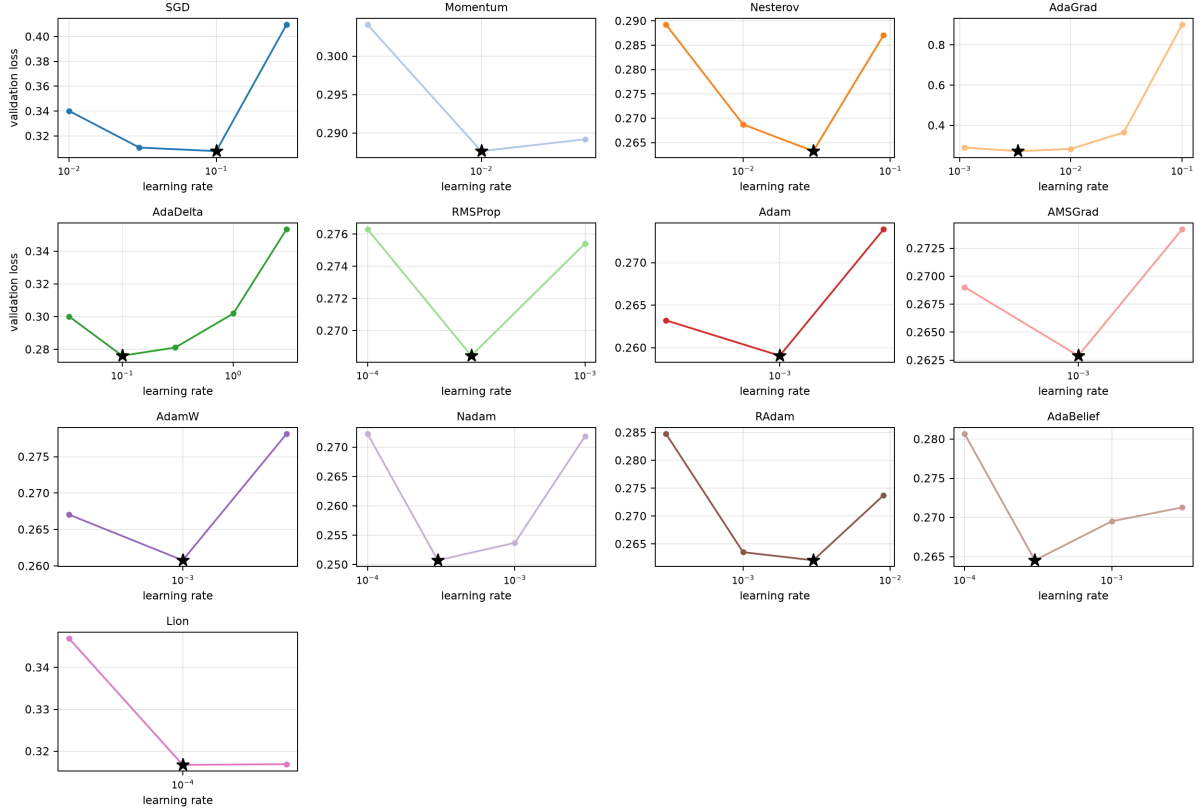


Figure 4: Learning-rate sweeps for the 13 optimizers. Each panel shows the validation loss after four epochs for each tested reference learning rate; the star marks the selected value.

training, and Figure 7 the final accuracy. Final-epoch results are reported consistently; no checkpoint is selected using test accuracy.

Optimizer	η_0	Train loss	Test loss	Test accuracy (%)
AdaBelief	0.0003	0.015	0.300	94.07 ± 0.12
Adam	0.001	0.015	0.330	94.03 ± 0.03
AMSGrad	0.001	0.015	0.296	94.03 ± 0.15
RMSProp	0.0003	0.016	0.314	94.03 ± 0.20
Momentum	0.01	0.023	0.266	94.02 ± 0.27
Nadam	0.0003	0.015	0.300	93.98 ± 0.14
RAdam	0.003	0.038	0.292	93.97 ± 0.06
SGD	0.1	0.025	0.265	93.93 ± 0.04
Lion	0.0001	0.019	0.300	93.93 ± 0.12
AdamW	0.001	0.015	0.326	93.91 ± 0.11
Nesterov	0.03	0.019	0.302	93.91 ± 0.10
AdaDelta	0.1	0.026	0.269	93.86 ± 0.19
AdaGrad	0.00333	0.057	0.207	93.73 ± 0.13

Table 1: FashionCNN results after 30 epochs with OneCycleLR (mean over 3 seeds), sorted by final test accuracy.

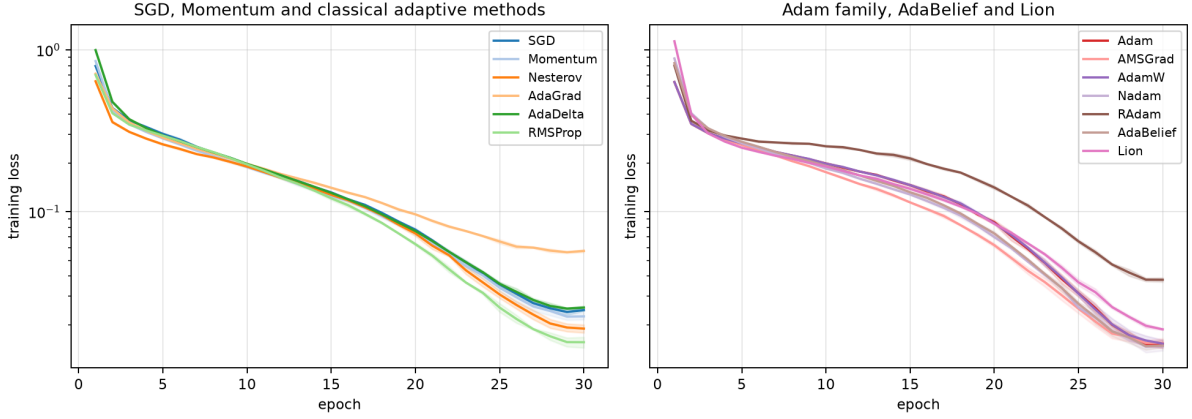


Figure 5: Example-weighted training loss per epoch, measured in training mode (mean over 3 seeds; the shaded area is one sample standard deviation).

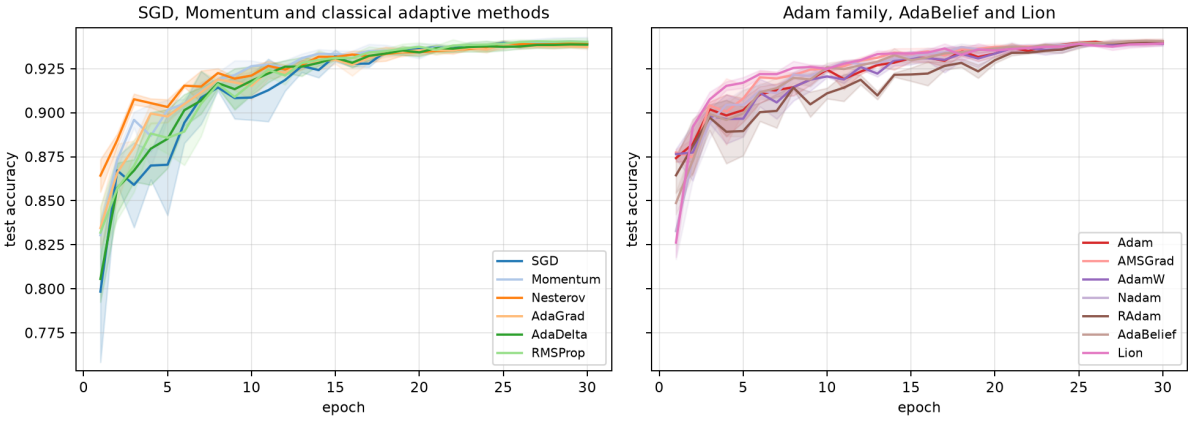


Figure 6: Test accuracy at the end of each epoch, measured in evaluation mode (mean over 3 seeds; the shaded area is one sample standard deviation).

Discussion. After 30 epochs, mean test accuracy ranges from 93.73% for AdaGrad to 94.07% for AdaBelief, a spread of 0.34 percentage points. AdaBelief has the highest observed mean ($(94.07 \pm 0.12)\%$); Adam, AMSGrad and RMSProp each reach about 94.03%, and Momentum reaches $(94.02 \pm 0.27)\%$. These are descriptive comparisons: sample standard deviations range from 0.03 to 0.27 percentage points, and three seeds do not establish a definitive ranking or a universal advantage for one optimizer family. Plain SGD reaches $(93.93 \pm 0.04)\%$.

The early optimization dynamics differ more noticeably than the final accuracies. Adam, AMSGrad and AdamW have the lowest first-epoch mean training loss (about 0.633), closely followed by Nesterov at 0.639; AdaDelta starts at 0.996. By epoch 30, Adam, AMSGrad, AdamW, Nadam and AdaBelief have mean training losses near 0.015, while Lion ends at 0.019 and AdaGrad at 0.057. Nevertheless, Lion reaches 93.93% test accuracy, close to the other methods, illustrating that a smaller training loss does not necessarily imply better final classification accuracy.

Accuracy and test cross-entropy also produce different rankings. AdaGrad has the lowest final test loss (0.207), despite the lowest mean accuracy (93.73%); SGD and Momentum have test losses of 0.265 and 0.266, respectively. AdaBelief has the highest mean

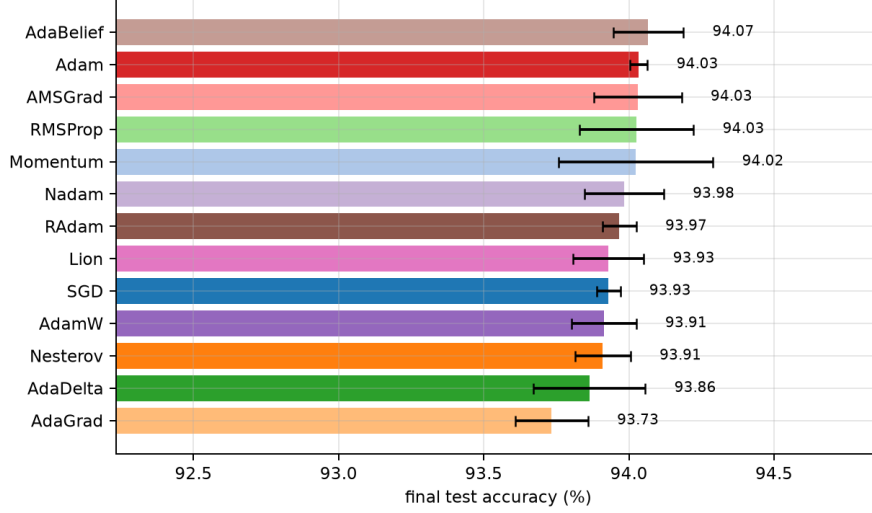


Figure 7: Test accuracy after epoch 30 (mean and sample standard deviation over 3 seeds). The horizontal axis is truncated; differences are measured in percentage points.

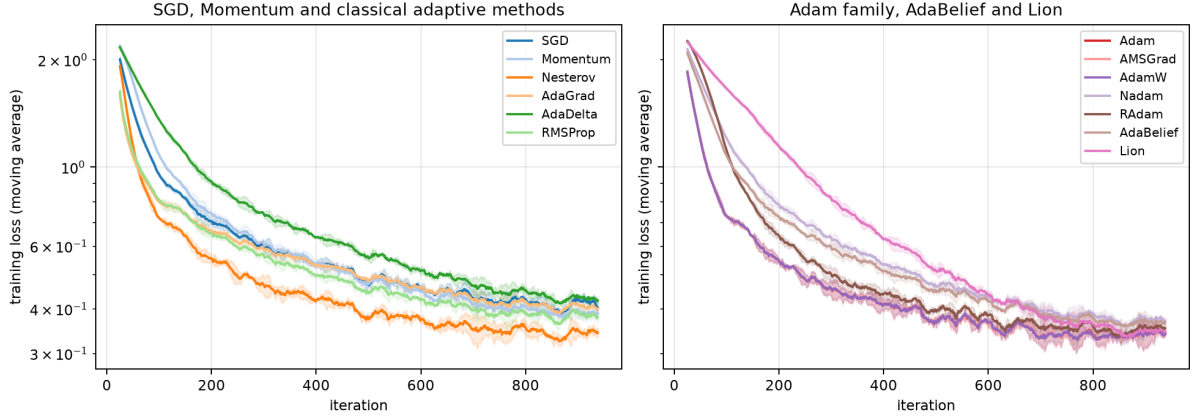


Figure 8: Mini-batch training loss during the first 938 updates (two epochs). A trailing moving average over 25 mini-batches is computed for each seed, then the mean and sample standard deviation over 3 seeds are plotted. The first displayed point ends at update 25.

accuracy with test loss 0.300, while Adam has test loss 0.330 at 94.03% accuracy. Cross-entropy depends on the probabilities assigned to labels, not just the most likely class; calibration itself is not measured in this experiment. The reported train and test losses differ substantially, but this difference alone is not a controlled estimate of a generalization gap: training loss averages predictions made with evolving weights, active dropout and mini-batch normalization, whereas test loss uses the final epoch weights with dropout disabled and running batch-normalization statistics. A direct comparison would require evaluating both training and test sets with the same fixed model in evaluation mode; those training-set evaluation losses are not saved.

Overall, all thirteen methods achieve similar final classification performance under this architecture, regularization and tuned OneCycleLR protocol. The experiment compares complete training configurations, not isolated update rules: both the optimizer and its selected learning-rate trajectory contribute to the result. Its scope is limited to Fashion-MNIST, FashionCNN, the 30-epoch budget, three seeds and a short four-epoch sweep.

The results should not be generalized beyond this model, dataset and training setup.

7 Conclusions

This paper reviewed first-order optimization methods for deep learning, from gradient descent and momentum to adaptive methods and recent variants of Adam. It also discussed large-scale and matrix-aware optimizers, techniques that complement the update rule, and second-order methods whose computational requirements can limit their use in large models. Finally, thirteen first-order optimizers were compared by training Fashion-CNN on Fashion-MNIST for 30 epochs with OneCycleLR and three random seeds, using native PyTorch implementations for eleven methods and notebook implementations for AdaBelief and Lion.

No optimizer is uniformly best across all settings. SGD with momentum is often used as a baseline in computer-vision studies, while AdamW is commonly used for Transformer training; these practices do not establish a universal ranking. In the experiment of Section 6, AdaBelief achieved the highest observed mean final accuracy (94.07%), but the difference between the highest and lowest means was only 0.34 percentage points. The close final accuracies, differing loss rankings, and limited number of seeds do not support interpreting this ordering as a universal hierarchy. Learning-rate selection, scheduling, regularization, and evaluation metrics should therefore be considered jointly. Further experiments across models and datasets, using more seeds and controlled ablations, are needed to distinguish their effects. For a broader discussion of adaptive methods, see [36].

References

- [1] Sutskever, I. et al. *On the Importance of Initialization and Momentum in Deep Learning*. <http://proceedings.mlr.press/v28/sutskever13.pdf>
- [2] Nesterov, Y. *A method for solving the convex programming problem with convergence rate $O(1/k^2)$* . Soviet Mathematics Doklady, 27(2), 372–376, 1983.
- [3] Johnson, R. and Zhang, T. *Accelerating Stochastic Gradient Descent using Predictive Variance Reduction*. Advances in Neural Information Processing Systems (NIPS), 2013.
- [4] Defazio, A., Bach, F. and Lacoste-Julien, S. *SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives*. <https://arxiv.org/abs/1407.0202>
- [5] Defazio, A. and Bottou, L. *On the Ineffectiveness of Variance Reduced Optimization for Deep Learning*. <https://arxiv.org/abs/1812.04529>
- [6] Duchi, J. et al. *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*. <http://www.jmlr.org/papers/volume12/duchi11a/duchi11a.pdf>
- [7] Hinton, G. *RMSPProp – Lecture 6e, Coursera Neural Networks for Machine Learning*. http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

- [8] Zeiler, M. D. *ADADELTA: An Adaptive Learning Rate Method*. <https://arxiv.org/abs/1212.5701>
- [9] Kingma, D. P. and Ba, J. *Adam: A Method for Stochastic Optimization*. <https://arxiv.org/abs/1412.6980>
- [10] Reddi, S. J., Kale, S. and Kumar, S. *On the Convergence of Adam and Beyond*. <https://arxiv.org/abs/1904.09237>
- [11] Dozat, T. *Incorporating Nesterov Momentum into Adam*. ICLR Workshop, 2016.
- [12] Loshchilov, I. and Hutter, F. *Decoupled Weight Decay Regularization*. <https://arxiv.org/abs/1711.05101>
- [13] Liu, L. et al. *On the Variance of the Adaptive Learning Rate and Beyond*. <https://arxiv.org/abs/1908.03265>
- [14] Zhuang, J. et al. *AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients*. <https://arxiv.org/abs/2010.07468>
- [15] You, Y., Gitman, I. and Ginsburg, B. *Large Batch Training of Convolutional Networks*. <https://arxiv.org/abs/1708.03888>
- [16] You, Y. et al. *Large Batch Optimization for Deep Learning: Training BERT in 76 minutes*. <https://arxiv.org/abs/1904.00962>
- [17] Shazeer, N. and Stern, M. *Adafactor: Adaptive Learning Rates with Sublinear Memory Cost*. <https://arxiv.org/abs/1804.04235>
- [18] Chen, X. et al. *Symbolic Discovery of Optimization Algorithms*. <https://arxiv.org/abs/2302.06675>
- [19] Gupta, V., Koren, T. and Singer, Y. *Shampoo: Preconditioned Stochastic Tensor Optimization*. <https://arxiv.org/abs/1802.09568>
- [20] Vyas, N. et al. *SOAP: Improving and Stabilizing Shampoo using Adam*. <https://arxiv.org/abs/2409.11321>
- [21] Jordan, K. *Muon: An optimizer for hidden layers in neural networks*. <https://kellerjordan.github.io/posts/muon/>
- [22] Liu, H. et al. *Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training*. <https://arxiv.org/abs/2305.14342>
- [23] Loshchilov, I. and Hutter, F. *SGDR: Stochastic Gradient Descent with Warm Restarts*. <https://arxiv.org/abs/1608.03983>
- [24] Goyal, P. et al. *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour*. <https://arxiv.org/abs/1706.02677>
- [25] Smith, L. N. *Cyclical Learning Rates for Training Neural Networks*. <https://arxiv.org/abs/1506.01186>
- [26] Pascanu, R., Mikolov, T. and Bengio, Y. *On the difficulty of training recurrent neural networks*. <https://arxiv.org/abs/1211.5063>

- [27] Glorot, X. and Bengio, Y. *Understanding the difficulty of training deep feedforward neural networks*. AISTATS, 2010.
- [28] He, K. et al. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. <https://arxiv.org/abs/1502.01852>
- [29] Zhang, M. R. et al. *Lookahead Optimizer: k steps forward, 1 step back*. <https://arxiv.org/abs/1907.08610>
- [30] Izmailov, P. et al. *Averaging Weights Leads to Wider Optima and Better Generalization*. <https://arxiv.org/abs/1803.05407>
- [31] Foret, P. et al. *Sharpness-Aware Minimization for Efficiently Improving Generalization*. <https://arxiv.org/abs/2010.01412>
- [32] Liu, D. C. and Nocedal, J. *On the limited memory BFGS method for large scale optimization*. Mathematical Programming, 45, 503–528, 1989.
- [33] Martens, J. *Deep learning via Hessian-free optimization*. ICML, 2010.
- [34] Martens, J. and Grosse, R. *Optimizing Neural Networks with Kronecker-factored Approximate Curvature*. <https://arxiv.org/abs/1503.05671>
- [35] Xiao, H., Rasul, K. and Vollgraf, R. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. <https://arxiv.org/abs/1708.07747>
- [36] Wilson, A. C. et al. *The Marginal Value of Adaptive Gradient Methods in Machine Learning*. <https://arxiv.org/abs/1705.08292>